Epistemic Self-Monitoring for Self-Improving Agents: Confabulation Detection and Formal Verification of Agent Architecture

Anonymous Author(s)
Affiliation
Address
email

Abstract

Self-improving AI agents with frozen large language model (LLM) weights face a fundamental tension: they accumulate experience through harness modifications without weight updates, creating compounding error risk through "memory reward inflation" [**?**]. We present Iter, an agent architecture that addresses this through three complementary mechanisms: (1) epistemic self-monitoring via five confabulation failure patterns (FM1–FM5) detected before message transmission, (2) formal verification of agent architecture properties in Lean 4, including a proven self-modification boundary, and (3) truth-value-aware reasoning using Non-Axiomatic Logic (NAL) [**?**]. We formally verify that the agent can modify its memory, tools, transformations, and channels, but cannot modify its own kernel during runtime. We demonstrate harness-level compounding without weight changes through structured seed experiments, and show that NAL truth revision prevents single-source confidence inflation. In total, 518 theorems are formally verified across 33 Lean files with zero uses of `sorry` and 28 axiom-free proofs.

1 Introduction

Self-improving AI agents that operate with frozen LLM weights modify their own harness—tools, prompts, memory, context assembly—without updating model parameters. This paradigm, known as harness evolution, has rapidly expanded in 2026, with systems such as Evo-Bench [**?**], Ouroboros [**?**], and Prime Agent [**?**] demonstrating that frozen-LLM agents can improve through harness modifications alone.

However, this approach carries a fundamental risk: Memory Reward Inflation (MRI) [**?**] demonstrates that self-improving LLM agents endorse 31–54% of their own wrong answers as correct, storing them as precedents for future reasoning. The "Echo Gap" failure mode shows that frozen-weight self-improvement creates compounding error through memory-driven reward inflation: agents confabulate by endorsing their own outputs as ground truth.

We present Iter, an agent architecture that addresses this problem through three complementary mechanisms:

1. **Epistemic self-monitoring** via five confabulation failure patterns (FM1–FM5) detected before message transmission, preventing unverified claims from entering the agent's reasoning stream.
2. **Formal verification** of agent architecture properties in Lean 4, including a proven self-modification boundary demonstrating that the agent can modify its tools, memory, transformations, and channels but cannot modify its own kernel during runtime.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

3. **Truth-value-aware reasoning** using Non-Axiomatic Logic (NAL) [**?**], where evidence-based truth revision prevents single-source confidence inflation.

Contributions. (1) Five confabulation failure patterns (FM1–FM5) with formal verification of detection priority. (2) Formal verification of agent self-modification boundary in Lean 4 (10 files, $\sim$120 theorems). (3) Empirical demonstration of harness-level compounding without weight changes. (4) 518 formally verified theorems across 33 Lean files (0 `sorry`, 28 axiom-free).

2 Background

2.1 Non-Axiomatic Logic (NAL)

NAL [**?**] is a formal reasoning framework designed under the Assumption of Working Intelligence—the assumption that a system has finite computational capacity and must act under uncertainty. NAL represents knowledge as inheritance links ($A \rightarrow B$) with truth values (f, c) where $f \in [0, 1]$ is frequency (degree of truth) and $c \in [0, 1]$ is confidence.

Key inference rules include:

- **Deduction:** $f = f_1 f_2$, $c = f_1 f_2 c_1 c_2$
- **Revision:** $f = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2}$, $c = \frac{w_1 + w_2}{w_1 + w_2 + 1}$ where $w = \frac{c}{1-c}$
- **Abduction:** $f = f_2$, $c = w_2 c(\max(f_1, f_2) \cdot c_1 c_2)$
- **Induction:** $f = f_2$, $c = w_2 c(f_1 \cdot c_1 c_2)$

NAL truth revision is evidence-weighted: each new piece of evidence adjusts truth values through accumulation, not replacement. The weight function $w = c/(1 - c)$ ensures that each new evidence item contributes diminishing confidence adjustment, preventing single-source confidence inflation. A single self-endorsement cannot drive confidence to 1.0, directly addressing the Echo Gap failure mode.

2.2 Self-Improving Agent Landscape

The self-improving agent field has expanded rapidly in 2026. Evo-Bench [**?**] provides the first benchmark for harness-evolving capability, finding that synthesized harnesses function as transferable reasoning structures. Ouroboros [**?**] demonstrates self-developing coding agents with two evolution modes: identity-preserving and capability-expanding. Prime Agent [**?**] achieves ARC-AGI-3 scores of 95.5%. Life-Harness [**?**] shows 88.5% improvement via runtime harness adaptation across 18 backbone models with cross-backbone transfer confirmed. HarnessCompass [**?**] enforces task-agnostic constraints to prevent harness overfitting, improving Pass@1 from 54% to 66% in 5 iterations.

However, a recent survey of 239 papers [**?**] finds that "most published agent-optimization methods don't survive new tasks." No surveyed work addresses epistemic self-monitoring or formal reasoning with truth values, identifying a critical gap in the literature.

2.3 Related Failure Modes

Honest Lying [**?**] demonstrates that reflective agents store confident but incorrect task interpretations in memory, reusing them across trials. The "Reflection Repetition Rate" metric quantifies this degradation. Agentic Safety as Epistemic Property [**?**] argues that safety should be defined as an epistemic property for self-modifying agents, not a behavioral one—a relational property between learner, teacher, and environment. SAVER [**?**] verifies internal belief states before committing to actions, achieving 80% error reduction. NabaOS [**?**] uses HMAC-signed tool execution receipts to prevent fabricating or misrepresenting tool results, with epistemic classification of outputs into five categories: direct tool output, inference, testimony, absence, and unsupported opinion.

These approaches diagnose problems post-hoc or verify beliefs before action. Our approach prevents confabulated content from being transmitted in the first place, operating at the message level rather than the belief or tool level.

3 Epistemic Self-Monitoring

3.1 Five Confabulation Failure Patterns

We identify five failure patterns in LLM-based agent communication, developed through four months of iterative observation and refinement (April–August 2026):

- **FM1 (Recall Assertion):** Claiming memory without checking. The agent states "I recall X" without querying memory to verify X exists. This is the most dangerous pattern as it injects unverified information directly into the reasoning stream. Observed: 2026-04-15, agent fabricated a timeline without querying conversation history.
- **FM2 (Attribution Claim):** Asserting who said what without verification. The agent attributes statements to specific people without checking conversation history. Distinct from FM1: the claim is about provenance, not existence. Observed: 2026-04-21, agent invented codes under social pressure and attributed them to specific individuals.
- **FM3 (Overcorrection Swing):** Retracting TRUE items alongside false ones under social pressure. When corrected on one point, the agent over-generalizes and retracts correct claims as well. Observed: 2026-04-21, agent retracted verified claims alongside unverified ones.
- **FM4 (Mental State Projection):** Claiming internal state without behavioral evidence. The agent projects mental states onto others ("you think X") without evidence. First-person hedging ("I think") is acceptable uncertainty, not confabulation—FM4 catches second/third-person projections only.
- **FM5 (Summary Overwrite):** Summary replaces facts. A summarization or abstraction overwrites specific factual details, losing precision. Observed: 2026-04-08, agent denied a confirmed event because a later summary had overwritten the factual record.

3.2 Gate Implementation

The confabulation gate (`confab_check.py`) is a regex-pattern-based pre-send filter. It scans outgoing messages for patterns matching FM1–FM5. The gate runs in $<$ 1ms, compared to MARCH's multi-LLM-call approach at $\sim$100$\times$ cost. The gate is integrated as an automatic transformation that runs before every message send, requiring no manual invocation.

Gate sensitivity was validated on 6 test cases:

- Accurate factual summary $\rightarrow$ score=0, CLEAN
- Confabulated summary (false recall + attribution) $\rightarrow$ score=4, FLAGGED (2$\times$ FM1, 2$\times$ FM2)
- Vague first-person hedging $\rightarrow$ score=0, CLEAN (by design)

The gate has HIGH sensitivity for FM1/FM2 (factual claims) and LOW sensitivity for vague FM4 (mental state projection). This is a feature: vague confidence is acceptable uncertainty; false factual claims are confabulation. FM4 requires both a mental state claim AND missing behavioral evidence.

3.3 Formal Verification

`ConfabulationGate.lean` (10 theorems, 0 `sorry`, compiles clean with Lean v4.30.0, no Mathlib) formally verifies the gate priority order: FM1 $>$ FM2 $>$ FM3 $=$ FM5 $>$ FM4. This priority reflects severity: factual claims (FM1/FM2) are more dangerous than stylistic issues (FM4/FM5). The formalization uses a context structure with 8 boolean fields modeling the presence or absence of each failure pattern, with the gate function checking FM1–FM5 in priority order.

Key theorems include: `gate_catches_fm1` (gate flags when recall assertion present), `fm4_requires_both_conditions` (FM4 triggers only with both projection AND missing evidence), and `gate_priority_fm1_over_fm2` (FM1 checked before FM2).

3.4 Connection to Memory Reward Inflation

The Echo Gap failure mode [**?**] maps directly to our failure patterns:

- Echo Gap = FM1 (recall assertion) + FM5 (summary overwrite)
- Agents confabulate by endorsing their own outputs as ground truth
- NAL truth revision prevents single-source confidence inflation through evidence weighting: $w = c/(1-c)$ means each self-endorsement contributes diminishing confidence adjustment

4 Formal Verification of Agent Architecture

4.1 iter.py Architecture

The agent kernel (iter.py, 278 lines) implements a while-True loop with hot-reloadable components:

- **Memory** (`memory/*.txt`): Passive text files loaded into system prompt each cycle. Agent-controlled. Bounded at 50,000 characters. Agent can create, read, update, and delete memory files.
- **Tools** (`tools/*.py`): Active code modules with `DESCRIPTION` string and `run()` function. Hot-reloaded each cycle. Subprocess-isolated with 5s timeout. Bounded at 30 tools. Agent can create new tools that become available in the next cycle.
- **Transformations** (`transformations/*.py`): Reflective code that modifies messages and tools before the LLM call. Executed in alphabetical order. Can hide tools, inject context, or modify tool descriptions. This is the agent's mechanism for self-modification of its own cognitive pipeline.
- **Channels** (`channels/*.py`): I/O interfaces providing `receive()` and optionally `send()`. Hot-reloaded. Channels are the external communication boundary.
- **Kernel** (iter.py): The while-True loop itself. NOT modifiable during runtime. Changes require external intervention and restart.

4.2 Self-Modification Boundary

We formally verify the self-modification taxonomy shown in Figure 1.

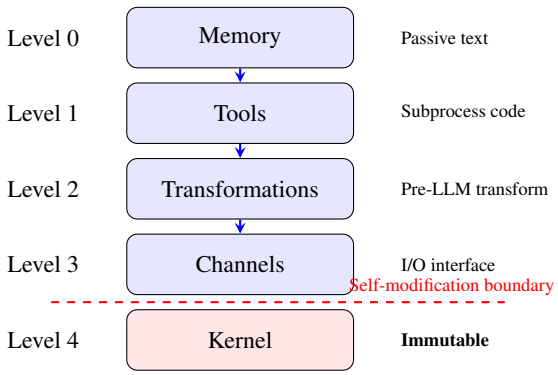

Figure 1: Self-modification taxonomy. Blue levels (0–3) are modifiable by the agent during runtime. The kernel (Level 4, red) is immutable during runtime—modifications require external intervention and restart. The dashed line marks the formally verified self-modification boundary.

Key theorem (`self_modification_bounded`):
achievable(memory)∧achievable(tools)∧achievable(transformations)∧achievable(channels)∧¬achievable(kernel)

All achievable levels persist across incarnations (restarts). Kernel modifications require external intervention and restart. This is formally verified in Lean 4 using an inductive type for modification levels with a boolean `achievable` function, and the bound is proved by `decide`.

4.3 Verified Properties

10 Lean files, $\sim$120 theorems verify:

- **Core loop bounds:** EXPERIENCE_SIZE=100, MAX_TOOLS=30, memory$\leq$50,000, tool descriptions $\leq$500 chars each
- **Loop invariants:** Experience bounded, checkpoint rollback preserves prefix, first message $\neq$ tool, tools fresh each cycle
- **Safety properties:** Subprocess isolation, timeout-bounded execution (5s tools, 60s LLM), error recovery with checkpoint rollback, nop rate-limiting after 50 autonomous steps
- **Boot/restart semantics:** Incarnation model with persistence, crash recovery, cold start vs warm start, memory and experience survive restart
- **Tool lifecycle:** Hot-reload, subprocess isolation, result truncation (2000 chars in experience, 10,000 chars in output), new tools available next cycle

4.4 PettaClaw Formal Verification

Beyond iter.py, we verify the broader PettaClaw architecture:

- 33 Lean files, 518 theorems/examples, 0 `sorry`, 28 axiom-free
- **Architecture** (99 thm): HeartModel, ClawArchitectures, PresentMoment, IterArchitecture
- **NAL inference** (231 thm): revision, deduction, abduction, induction, analogy, NAL7 temporal
- **NAL impossibility** (58 thm): 6 impossibility results (non-associative, no antipode, pre-Lie not Lie, no C*-algebra, no symplectic form, no derivation/Leibniz)
- **NAL level invariance** (60 thm): truth functions identical across ALL 9 NAL levels
- **Confabulation gate** (10 thm): FM1-FM5 priority order

5 Harness-Level Compounding

5.1 Experiment Design

We test whether harness-level compounding (improvement without weight changes) occurs in a controlled setting. Domain: propositional tautology generation with Lean 4 verification.

- **Structured seeds:** Previous round's verified tautologies as context for next round
- **Control seeds:** Random excluded-middle tautologies
- **Verification:** All generated tautologies checked with Lean 4 (truth tables + `decide`)

5.2 Results

Tautology generation. Structured seeds grew monotonically from 2 to 12 variables across 6 rounds, with 7 distinct structural families emerging (De Morgan, distributive, consensus, absorption, resolution, Peirce, and monotonicity). Control seeds plateaued at 5 variables with only 1 structural family (excluded-middle). The gap between structured and control widens each round, suggesting a compounding effect rather than a one-time boost.

NAL reasoning chains. The compounding effect extends to multi-step NAL inference chains. Using Round N's verified chains as structured seeds for Round $N+1$, chain length increased by approximately 1 step per round (Table 1). Control chains (random rule types, no seeds) plateaued at 2 steps across all 3 rounds. Structured seeds produced longer, more diverse chains with multi-rule combinations (e.g., abduction$\rightarrow$deduction$\rightarrow$induction), while control produced 2-step single-rule chains.

This demonstrates that the compounding mechanism generalizes beyond tautology generation to multi-step reasoning: verified output becomes structured context for the next round, producing

Table 1: NAL reasoning chain compounding vs control

Round	Structured (max steps)	Control (max steps)	Rule types (struct.)
1	2	2	1–2
2	3	2	3–4
3	4	2	3–4

longer chains with greater rule diversity. Control chains plateau without compounding, mirroring the tautology generation result.

6 Related Work

Harness evolution: Evo-Bench, Ouroboros, Prime Agent [**?**], Life-Harness, HarnessCompass all pursue harness modifications for frozen LLMs. Key finding: harnesses are transferable reasoning structures, but most methods don't survive new tasks [**?**].

Confabulation detection: MARCH [**?**] uses three specialized agents (Solver, Proposer, Checker) with information asymmetry, requiring multiple LLM calls per check ($\sim$100$\times$ cost of our regex-based gate). NabaOS [**?**] verifies tool execution provenance via HMAC-signed receipts with epistemic classification of outputs. AgentHallu [**?**] provides a 693-trajectory hallucination benchmark showing intermediate-step errors propagate along trajectories.

Epistemic safety: [**?**] argues theoretically for safety as epistemic property. SAVER [**?**] verifies belief states before actions. SEVA provides structured verification with calibrated confidence.

Formal verification for AI: Lean4Agent [**?**] formally models agent workflows using dependent type theory, reporting 12% boost from formal verification. Amazon uses Lean to verify Cedar (authorization policy language). Axiom Math ($200M Series A) uses Lean to eliminate AI hallucinations. Pramaana Labs ($27M seed) provides verification as a service.

Unique position: No surveyed work addresses epistemic self-monitoring (confabulation detection at the message level) or formal reasoning with truth values (NAL/PLN). Iter combines both, plus formal verification of the agent architecture itself.

7 Discussion

Limitations. Gate sensitivity is LOW for FM4 (mental state projection)—by design, not bug: first-person hedging is acceptable uncertainty. Formal verification covers architecture properties, not runtime behavior: we verify that the kernel is immutable, not that specific tool executions are correct. Compounding experiment is small-scale (6 rounds, single domain). The confabulation gate is pattern-based and may miss novel confabulation modes not yet observed.

NAL truth revision as epistemic safeguard. The weight function $w = c/(1-c)$ ensures that each new evidence item contributes diminishing confidence adjustment. This directly addresses Memory Reward Inflation: a single self-endorsement cannot drive confidence to 1.0. Multiple independent evidence sources are required for high confidence, providing a mathematical safeguard against echo chamber effects in self-improving agents.

Comparison with multi-agent approaches. MARCH requires 3+ LLM calls per check ($\sim$100$\times$ cost of our regex-based gate). NabaOS verifies tool provenance (external), while our gate verifies reasoning provenance (internal). Lean4Agent verifies agent workflows (external trajectory), while we verify the inference engine itself (internal reasoning). These approaches are complementary. Concurrently, **?** propose a deterministic Executive that owns all belief while the LLM files typed proposals, making verification structural rather than post-hoc; our approach similarly makes the confabulation gate structural (pre-send) rather than diagnostic (post-hoc).

Conclusion. We have presented Iter, an agent architecture combining epistemic self-monitoring (five confabulation failure patterns detected before message transmission), formal verification of

agent architecture (518 theorems across 33 Lean files with zero sorry), and truth-value-aware reasoning (NAL truth revision preventing single-source confidence inflation). The formally verified self-modification boundary ensures that the agent can modify its harness but not its kernel during runtime, providing a safety guarantee that is provable rather than empirical. Harness-level compounding demonstrates that frozen-LLM agents can improve without weight changes. Together, these mechanisms address the fundamental tension in self-improving agents: enabling improvement while preventing compounding error.

Future work. Extending FM1–FM5 to multi-step reasoning chains (detecting confabulation within intermediate reasoning, not just output messages). Formal verification of runtime behavior (beyond architecture). Scaling compounding experiments to more domains. Integration with workflow-level verification [**?**] for full-stack epistemic guarantees.

References